UAI301 Object Oriented Programming

L	T	P	Cr
3	**0**	2	**4.0**

Course Overview: This course introduces the principles of Object-Oriented Programming using Python. Students will learn how to design and implement reusable, modular, and scalable code using classes, objects, inheritance, polymorphism, encapsulation, and other OOP concepts.

Introduction to OOP Concepts: What is Object-Oriented Programming? Difference between procedural and object-oriented programming, Core OOP principles: Encapsulation, Abstraction, Inheritance, Polymorphism, Classes vs. Objects: Definitions and examples, Creating your first class in Python.

Working with Classes and Objects: Defining attributes and methods, The __init__ method (constructor), Instance variables vs. class variables, Accessing and modifying object properties, Understanding self keyword, Deleting objects and attributes. Case Study on Optimizing Power Consumption in Smart City Energy Grids.

Encapsulation and Access Modifiers: Public, protected, and private members, Name mangling in Python, Getters and setters, Data hiding and abstraction

Inheritance: Types of inheritance: single, multiple, multilevel, hierarchical, Base and derived classes, Method overriding, Use of super() function, isinstance() and issubclass() functions.

Polymorphism: Method overloading (simulation in Python), Operator overloading, Duck typing and polymorphic behavior, Polymorphism with inheritance and interfaces. Case Study on Developing an Eco-Friendly Smart Transportation System.

Advanced OOP Concepts: Class methods and static methods (@classmethod, @staticmethod), Abstract base classes using abc module, Magic methods (__str__, __repr__, __add__, etc.), Composition vs. inheritance, Managing complex projects with modules and packages.

Error Handling & Debugging in OOP: Custom exceptions in classes, Exception handling inside methods, Logging errors and debugging OOP code, PEP8 guidelines for OOP.

Laboratory Work:

To implement object-oriented constructs using the Python programming language.

Course Learning Objectives (CLO)

The students will be able to:

1. To recall the knowledge of structure and its variables to comprehend the concept of classes, objects, constructors and destructors for implementing the object oriented paradigms.
2. To apply and analyze the inheritance on real life case studies via coding competences.
3. To design and develop code snippets for polymorphism to proclaim coding potential; and management of run-time exceptions.
4. To assess and interpret the knowledge of templates to appraise the standard template libraries.

Text Books:

1. Phillips, D. (2015). Python 3 Object-Oriented Programming (2nd ed.). Packt Publishing.
2. Matthes, E. (2019). Python Crash Course: A Hands-On, Project-Based Introduction to Programming . No Starch Press.

Reference Books:

1. Zelle, J. M. (2016). Python Programming: An Introduction to Computer Science (3rd ed.). Franklin, Beedle & Associates.
2. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.